MANAGING STATE IN AI AGENTS

Name: _____ **Branch/Year:** _____ **Date:** _____

Lesson Snapshot

Subject: AI Agents / LangGraph

Level: Intermediate (presented in beginner-friendly steps)

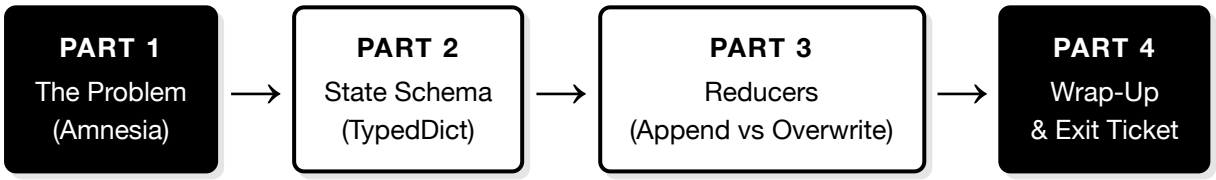
Duration: 50 Minutes

Prerequisites: LLMs, RAG, Basic Agent concepts

Learning Objectives (Tick when confident)

- [ ] I can define **State** and explain why LLMs are stateless by default.
- [ ] I can design a **State Schema** using `TypedDict`.
- [ ] I can explain and use **reducers**, especially `add_messages`.
- [ ] I can build a complete LangGraph agent with schema, nodes, and graph wiring.

Lecture Flow



Warm-Up: What Do You Already Know?

1. What best describes a "stateless" LLM?

A. It keeps all past conversations automatically forever.

B. It can only run one tool at a time.

C. It cannot answer follow-up questions without prior context.

D. It only remembers data from the current prompt unless memory is managed externally.

Hint: *Think about what happens when you close and reopen ChatGPT.*

2. When chat history is not stored, what goes wrong?

A. Slower internet speed

B. Python cannot import modules

C. The AI forgets everything between turns (amnesia)

D. The keyboard stops working

Hint: *Imagine telling someone your name, and they forget it 5 seconds later.*

3. Which of these fields should GROW over time (not be replaced)?

A. `task_status` (e.g., "pending" or "done")

B. `messages` (conversation history)

C. `current_price` (latest price of a product)

D. `is_verified` (True/False)

Hint: *Which one would you lose context if you replaced it?*

4. What Python module is used to define a strict "shape" for data?

A. `random`

B. `TypedDict` from `typing`

C. `os.path`

D. `json.dumps`

Hint: *It defines keys and their types before runtime.*

Part 1: The Problem - AI Amnesia

THE SCENARIO: "The Forgetful Food Delivery Bot"

Imagine you're ordering food through a chatbot:

Turn	You Say	Bot (With Memory)	Bot (No Memory)
1	"I want a pizza."	"Great! What size?"	"Great! What size?"
2	"Large, please."	"Large pizza. Toppings?"	"Large what?"
3	"Pepperoni."	"Large pepperoni pizza. Address?"	"Pepperoni? What are we talking about?"

Without state, every turn starts from zero. The bot has *amnesia*.

CONCEPT: A standard LLM call behaves like a pure function: `f(input) = output`. Each call starts fresh. The LLM has no built-in ability to remember previous calls unless we **explicitly manage memory**.

THE ANALOGY: The Goldfish vs. The Detective

Stateless LLM = A goldfish. It only "sees" what's in its current bowl (the prompt). Every new prompt is a brand-new bowl.

Agent with State = A detective. It keeps a case file that stores every clue, witness statement, and note from Day 1 onwards.

Situation	Stateless LLM	Agent with State
User shares their name once	Forgets next call	Remembers for the session
3-step booking workflow	Loses all previous steps	Keeps full context
Tool returns a result	Must re-send manually	Stored in state automatically
App crashes mid-task	Everything is lost	Resume from checkpoint

KEY INSIGHT: State is the snapshot of everything the agent knows *right now* — messages, user info, task progress, and tool results. It is the agent's memory.

5. A booking bot asks "What's your name?" on Turn 3, even though the user said it on Turn 1. What is the root cause?

A. The user typed too fast

B. The LLM is stateless and did not receive Turn 1 data in Turn 3's prompt

C. The internet connection dropped

D. Python ran out of memory

6. Before state management solutions, developers often handled context by:

A. Asking the user to repeat everything each turn

B. Re-sending the entire conversation history in every prompt (prompt stuffing)

C. Rebooting the server after every 5 messages

D. Using print statements to remember data

ACTIVITY: Think and Write: Name one real-world app where "AI memory" is essential, and explain why in one sentence.

TRANSITION: Now that we know **why** memory matters, let's define **what** memory looks like in code.

Part 2: Defining the State Schema

THE ANALOGY: The Architect's Blueprint

Before building a house, an architect draws a blueprint — it defines rooms, sizes, and purposes *before a single brick is laid*.

Similarly, a **State Schema** defines what fields (rooms) the agent's memory will have, and what type of data each field stores — *before the agent runs*.

CONCEPT: In LangGraph, we use Python's `TypedDict` to create the state schema. This gives us type safety and a clear contract for what data flows through the agent graph.

Step 1: Define the Schema (The Blank Form)

```
 # Step 1: Import the building blocks
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

# Step 2: Define the state schema (the "blank form")
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]   # Conversation history
    user_name: str                            # Who is the user?
    current_task: str                         # What is the agent doing?
    task_status: str                          # How far along is it?
```

This class does **NOT** do anything on its own. It only defines *what boxes exist* on the form and *what type of data* goes in each box. Think of it as a blank patient form at a hospital — it has fields for name, blood type, and medical history, but no data is filled in yet.

Field	Type	Purpose	Example Value
`messages`	`Annotated[list, add_messages]`	Full conversation history	`[HumanMessage('Hi'), AIMessage('Hello!')]`
`user_name`	`str`	User identity	`'Priya'`
`current_task`	`str`	Active workflow	`'book_flight'`
`task_status`	`str`	Progress tracking	`'payment_pending'`

Step 2: Define the Node Function (The Worker)

```
from langchain_core.messages import AIMessage

# The "worker" — reads the form, does work, writes back changes
def chat_node(state: AgentState):
    latest_text = state["messages"][-1].content  # READ the form
    reply = f"I heard: {latest_text}. How can I help?"

    return {                                      # WRITE back only what changed
        "messages": [AIMessage(content=reply)],
        "task_status": "in_progress"
    }
```

This function does **NOT** run on its own either. It is a worker waiting to be called. It reads the current state (the form), does some processing, and returns *only the fields it updates*.

WHY DO WE NEED BOTH? The Form + Worker + Manager

Students often ask: "*Why is there a class AND a function? Can't we just use one?*"

Part	Code	Real-World Analogy	What It Does
The Form (class)	`class AgentState(TypedDict)`	A blank patient form	Defines WHAT to remember (fields + types)
The Worker (function)	`def chat_node(state)`	The doctor	Reads form, does work, writes back changes
The Manager (LangGraph)	`app.invoke(...)`	The hospital office manager	Passes form to doctor, collects updates, saves form

You write the form (class) and the worker (function). LangGraph does everything else — loading state, calling your function, merging results, and saving.

Step 3: Connect Everything to the Graph

```
from langgraph.graph import StateGraph, START, END

# Tell LangGraph: "use this form (schema)"
graph = StateGraph(AgentState)

# Tell LangGraph: "this worker (function) is a node"
graph.add_node("chat", chat_node)

# Tell LangGraph: "start here, end here"
graph.add_edge(START, "chat")
graph.add_edge("chat", END)

# Compile into a runnable app
app = graph.compile()
```

Now when you call `app.invoke(...)`, LangGraph automatically: loads state → calls `chat_node` → merges its return values → saves state. You never call `chat_node()` yourself.

Step 4: See It in Action — What Happens When You Call `app.invoke("Hi")`?

```
from langchain_core.messages import HumanMessage

# This is the ONLY line you write to start a conversation:
result = app.invoke({"messages": [HumanMessage(content="Hi")]})
```

Behind the scenes, here's **exactly** what happens step by step:

Stage	Who Does It		
1. Before invoke	—	`[]` (empty)	`""` (empty)
2. User message added	LangGraph	`[Human("Hi")]`	`""`
3. `chat_node` receives state	LangGraph calls it	`[Human("Hi")]`	`""`
4. `chat_node` returns	Your function	`{"messages": [AI("I heard: Hi. How can I help?")], "task_status": "in_progress"}`	
5. LangGraph merges	LangGraph	`[Human("Hi"), AI("I heard: Hi...")]` (appended!)	`"in_progress"` (overwritten!)

```
# What you get back:
print(result["messages"][-1].content)
# Output: "I heard: Hi. How can I help?"

print(result["task_status"])
# Output: "in_progress"
```

KEY INSIGHT: One line of code (`app.invoke`), five things happened automatically. That is the power of LangGraph — you define the form and the worker, it handles the rest.

WARNING: **Common Mistake:** Returning the entire state from every node. Instead, return *only* the fields your node changes. LangGraph handles merging automatically.

7. What is the purpose of the `class AgentState(TypedDict)` ?

A. It runs the agent and sends messages to the LLM

B. It defines the structure (fields and types) of the agent's memory — like a blank form

C. It stores data in a SQL database permanently

D. It creates a REST API endpoint for the chatbot

8. What is the purpose of the node function (e.g., `def chat_node(state)`)?

A. It defines the form — what fields exist

B. It is the worker — reads the current state, does processing, and returns only the updated fields

C. It saves data to disk automatically

D. It creates the graph structure

9. Who calls the node function?

A. The developer calls it manually with `chat_node(state)`

B. Python's `import` statement triggers it

C. LangGraph calls it automatically when `app.invoke()` is used

D. The LLM decides when to call it

10. Which Python type best represents an "identity verified" status (yes/no)?

A. `float`

B. `dict`

C. `bool`

D. `bytes`

ACTIVITY: Design Challenge: Design a state schema for a *movie ticket booking bot*. List 5 fields with their types.

Example: `movie_name: str`

TRANSITION: The schema tells us **what** to store. Next question: **how** should each field update when new data arrives?

Part 3: Reducers - Append vs Overwrite

THE ANALOGY: Whiteboard vs. Notebook

Overwrite = A whiteboard. Every time you write something new, you erase what was there before. Only the latest value stays.

Append = A notebook. Every time you write something, you add a new page. All previous entries are preserved.

CONCEPT: A **reducer** is the rule that controls how a state field updates when a node returns new data. In Part 2, we already saw two behaviors in our schema — now let's name them formally:

Update Type	How It Works	Code (from Part 2)	Best For
Overwrite (default)	New value replaces old value completely	`task_status: str` (no annotation)	Status, counters, latest price
Append (reducer)	New items added to existing list	`messages: Annotated[list, add_messages]`	Conversation history, logs

Look back at the schema from Part 2 — the `Annotated[list, add_messages]` on `messages` IS the reducer. Fields without that annotation use the default overwrite.

Seeing the Difference Across Multiple Turns

In Part 2, we traced a single `app.invoke("Hi")`. Now let's see what happens over **2 full turns** — this is where reducers really matter:

Step	What Happens	(Append)	(Overwrite)
Start	App is initialized	`[]` (empty)	`"idle"`
Turn 1	User: "Hi" → AI: "Hello!"	`[Human("Hi"), AI("Hello!")]`	`"idle"` (no change)
Turn 2	User: "Book a flight" → AI: "Sure!"	`[Human("Hi"), AI("Hello!"), Human("Book.."), AI("Sure!")]`	`"booking_started"` (replaced!)

KEY INSIGHT: Notice: `messages` keeps growing (4 items after 2 turns). But `task_status` only ever holds **one value** — the latest. This is the difference between append (reducer) and overwrite (default).

11. A flight-price tracker needs to store only the *latest* price. Which behavior should you use?

A. Append (add_messages)

B. Overwrite (default)

C. Delete and recreate

D. Store in a separate database only

12. What happens if you apply `add_messages` to `task_status` instead of `messages`?

A. It works better and is recommended

B. It creates a growing list of all past statuses, wasting memory for no benefit

C. Python will crash immediately

D. The agent becomes faster

13. For a to-do list agent, which field should use APPEND behavior?

A. `current_filter` (e.g., "show completed only")

B. `todo_items` (list of all tasks added by the user)

C. `sort_order` (e.g., "by_date")

D. `user_theme` (e.g., "dark mode")

WARNING: Do not apply `add_messages` to every field! Use it **only** where history must be preserved (like conversation logs). For single-value fields like status or counters, the default overwrite is correct.

ACTIVITY: Classify Each Field: For a *customer support bot*, mark each field as APPEND or OVERWRITE and give a one-line reason.

Field	APPEND or OVERWRITE?	Why?
`messages`		
`ticket_status`		
`customer_name`		
`escalation_log`		
`priority_level`		

TRANSITION: Now we understand both the schema (what to store) and reducers (how to update). Let's wrap up and test your understanding.

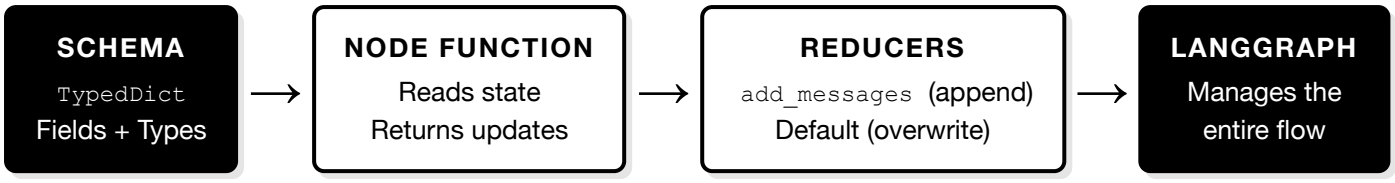
Part 4: Wrap-Up and Exit Ticket

KEY INSIGHT: State Management = Schema (what to store) + Node Functions (what to do) + Reducers (how to update).

Complete Summary Table

Component	Tool / Concept	One-Line Summary
The Form (Schema)	`class AgentState(TypedDict)`	Defines what memory fields exist and their types
The Worker (Node)	`def chat_node(state)`	Reads state, does work, returns only changed fields
The Manager	LangGraph / `app.invoke`	Loads state, calls workers, merges updates, saves
Append Reducer	`Annotated[list, add_messages]`	New messages added to list; old ones preserved
Overwrite (Default)	No annotation needed	Latest value replaces the old one

Architecture Overview



ACTIVITY: 3-Finger Confidence Check: Tick each concept you can explain in one sentence.

- [ ] Why stateless LLMs cause amnesia
- [ ] The Form (class) + Worker (function) + Manager (LangGraph) pattern
- [ ] `add_messages` (append) vs default (overwrite) reducers

Exit Ticket

14. A food delivery bot has these fields: `messages`, `order_status`, `delivery_address`. Which one uses an append reducer?

A. `order_status`

B. `messages`

C. `delivery_address`

D. All three

15. After 3 turns of conversation, how many items are in the `messages` list?

A. 3 (only user messages)

B. 6 (3 user + 3 AI messages)

C. 1 (only the latest message)

D. 0 (messages are not stored)

16. What happens when you call `app.invoke()`?

A. Nothing — you must also call `chat_node()` manually

B. LangGraph loads state, calls your node function, merges updates, and returns the result

C. Python prints "Hello World"

D. The schema is created for the first time

Complete Combined Code (All Steps Together)

This is a **Flight Ticket Booking Agent** that ties together all concepts covered in this lesson: state schema (with overwrite + append fields), node functions, graph wiring, and reducers.

```
 # ============================================================
# FLIGHT TICKET BOOKING AGENT
# Demonstrates: Schema, Nodes, Reducers, Graph Wiring
# ============================================================

# ── STEP 1: IMPORTS ──────────────────────────────────────────
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_core.messages import HumanMessage, AIMessage


# ── STEP 2: STATE SCHEMA (The Booking Form) ──────────────────
class BookingState(TypedDict):
    messages: Annotated[list, add_messages]  # Chat history      → APPEND
    passenger_name: str                      # Who is flying?    → OVERWRITE
    departure_city: str                      # From where?       → OVERWRITE
    arrival_city: str                        # To where?         → OVERWRITE
    travel_date: str                         # When?             → OVERWRITE
    booking_status: str                      # Current progress  → OVERWRITE


# ── STEP 3: NODE FUNCTIONS (The Workers) ─────────────────────
def collect_info_node(state: BookingState):
    """Greet the user and collect booking details from their message."""
    user_msg = state["messages"][-1].content if state["messages"] else ""

    # Simple keyword extraction (in real agent, an LLM would do this)
    name = "Priya" if "priya" in user_msg.lower() else state.get("passenger_name", "")
    departure = "Delhi" if "delhi" in user_msg.lower() else state.get("departure_city", "")
    arrival = "Mumbai" if "mumbai" in user_msg.lower() else state.get("arrival_city", "")
    date = "15 May" if "may" in user_msg.lower() else state.get("travel_date", "")

    reply = (f"Got it! Booking for {name}, "
             f"{departure} → {arrival} on {date}. Confirming...")

    return {
        "messages": [AIMessage(content=reply)],   # Appended by reducer
        "passenger_name": name,                    # Overwritten
        "departure_city": departure,               # Overwritten
        "arrival_city": arrival,                   # Overwritten
        "travel_date": date,                       # Overwritten
        "booking_status": "details_collected",     # Overwritten
    }


def confirm_booking_node(state: BookingState):
    """Confirm the booking and generate a PNR."""
    name = state.get("passenger_name", "Guest")
    dep = state.get("departure_city", "?")
    arr = state.get("arrival_city", "?")
    date = state.get("travel_date", "?")

    confirmation = (f"Booking CONFIRMED!\n"
                    f"Passenger: {name}\n"
                    f"Route: {dep} → {arr}\n"
                    f"Date: {date}\n"
                    f"PNR: SKY-98452")

    return {
        "messages": [AIMessage(content=confirmation)],  # Appended
        "booking_status": "confirmed",                  # Overwritten
    }


# ── STEP 4: BUILD THE GRAPH ──────────────────────────────────
graph = StateGraph(BookingState)

graph.add_node("collect_info", collect_info_node)
```

```
graph.add_node("confirm", confirm_booking_node)

graph.add_edge(START, "collect_info")        # Start → Collect
graph.add_edge("collect_info", "confirm")    # Collect → Confirm
graph.add_edge("confirm", END)               # Confirm → End

app = graph.compile()


# ─── STEP 5: INVOKE ──────────────────────────────────────────
result = app.invoke({
    "messages": [HumanMessage(
        content="Hi, I'm Priya. Book Delhi to Mumbai on 15 May."
    )]
})

# ─── OUTPUT ──────────────────────────────────────────────────
for msg in result["messages"]:
    role = "USER" if isinstance(msg, HumanMessage) else "BOT"
    print(f"[{role}] {msg.content}")
print(f"\nBooking Status: {result['booking_status']}")
print(f"Passenger: {result['passenger_name']}")
print(f"Route: {result['departure_city']} → {result['arrival_city']}")
print(f"Date: {result['travel_date']}")
```

KEY INSIGHT: This agent uses **2 nodes** (collect_info → confirm) connected in a pipeline. The `messages` field grows with every node (append reducer), while fields like `booking_status` always hold only the latest value (overwrite). This is the core pattern for all LangGraph agents.

Quality Checklist

#	Quality Criterion	Check
1	Clear objectives / success criteria	[ ]
2	Start-middle-end structure	[ ]
3	Logical topic sequencing	[ ]
4	One concept at a time	[ ]
5	Step-by-step subtopic breakdown	[ ]
6	Clear transitions between concepts	[ ]
7	Recap and repetition moments	[ ]
8	Simplified, relevant explanations	[ ]
9	Misconceptions proactively addressed	[ ]
10	Lecture aligned to assignment	[ ]
11	Measurable learning checks	[ ]
12	Use of practical examples	[ ]
13	Pause / reflection time included	[ ]
14	Pacing contingencies included	[ ]